

Lesson 8 — Sound Design With Web Audio

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

The graph

oscillator → gain → masterGain → speakers
(tone) (envelope) (volume)

```
const audio = new AudioContext();
const master = audio.createGain();
master.gain.value = 0.3; // headroom
master.connect(audio.destination);
```

One sound

```
function beep(freq, when, length) {
  const osc = audio.createOscillator();
  const g = audio.createGain();
  osc.type = "square";
  osc.frequency.setValueAtTime(freq, when);
  osc.connect(g);
  g.connect(master);

  // THE ENVELOPE. Without it: a click.
  g.gain.setValueAtTime(0, when);
  g.gain.linearRampToValueAtTime(1, when + 0.005);

  g.gain.exponentialRampToValueAtTime(0.0001,
  when + length);

  osc.start(when);
  osc.stop(when + length);
}
```

A new oscillator every time. They are one-shot and cheap.

`exponentialRampToValueAtTime(0, ...)` **throws** — you cannot reach zero by multiplying. Use `0.0001`.

The autoplay rule

Silence, no error → check `audio.state`.

```
function unlock() {
  if (audio.state === "suspended")
    audio.resume();
}
addEventListener("pointerdown", unlock);
addEventListener("keydown", unlock);
```

Timing

```
const when = audio.currentTime + 0.5; // exact
```

Never `setTimeout` for anything rhythmic. 15 ms late is audible; 15 ms of visual lag is not.

```
while (nextNote < audio.currentTime + 0.15) {
  playNote(melody[step], nextNote);
  nextNote += STEP_SECONDS; // never drifts
  step = (step + 1) % melody.length;
}
```

Noise

```
const buf = audio.createBuffer(1, n,
audio.sampleRate);
const d = buf.getChannelData(0);
for (let i = 0; i < n; i++)
  d[i] = Math.random() * 2 - 1;
```

noise + lowpass sweeping down = **explosion** noise + highpass = **wind** noise + very short envelope = **snare, footstep**

Notes from numbers

```
freq = 440 * Math.pow(2, semitones / 12);
```

Doubling = one octave. 12 semitones = doubling.

Shapes

sine soft · square retro · sawtooth harsh · triangle between

Section A — Recall

A1. What is a *sample*, and roughly how many are sent to the speaker each second? [2 marks]

A2. Why does a sound with no envelope click? Explain in terms of what the speaker is being asked to do. [3 marks]

A3. Name the four nodes in the chain from tone to speaker, in order. [2 marks]

A4. Your game is completely silent and the console is empty. What is the first thing to check, and why does this problem exist? [3 marks]

A5. Why must music be scheduled against `audio.currentTime` rather than driven from the game loop? [3 marks]
Why is the same wobble harmless for graphics?

A6. Why can `exponentialRampToValueAtTime` not ramp to exactly 0? [2 marks]

A7. What is clipping, and name two things that reduce it. [2 marks]

Section B — Predict the output

B1. A note is 220 Hz. What frequency is one octave above it? Two octaves? Seven semitones above 440 Hz, to the nearest hertz? [3 marks]

B2. How many samples are in a 0.25-second noise buffer at a sample rate of 48,000? [3 marks]

B3. The melody has 8 steps and `STEP_SECONDS` is 0.125. `audio.currentTime` is 2.00 and `nextNote` is 2.05, with a look-ahead of 0.15. How many notes get scheduled this frame, and at what times? [4 marks]

B4. Describe what the player hears. [3 marks]

```
osc.start(audio.currentTime);
osc.stop(audio.currentTime + 0.2);
// no gain envelope at all
```

B5. Ten sounds fire on the same frame, each with a gain of 1, into a master gain of 1. What happens, and what would a master gain of 0.3 change? [3 marks]

Section C — Find and fix the bug

C1. The first sound in the game never plays. Every sound after it is fine. [3 marks]

```
const audio = new AudioContext();
window.addEventListener("keydown", function () { jump(); });
```

C2. The console says `Failed to execute 'exponentialRampToValueAtTime'`. [3 marks]

```
g.gain.setValueAtTime(1, when);
g.gain.exponentialRampToValueAtTime(0, when + 0.3);
```

C3. The first beep works and every later one throws `cannot call start more than once`. [4 marks]

```
const osc = audio.createOscillator();
osc.connect(master);

function beep() {
  osc.frequency.value = 600;
  osc.start();
  osc.stop(audio.currentTime + 0.1);
}
```

C4. The music is in time while nothing else is happening, and falls apart whenever the game is busy. [4 marks]

Name the bug and the fix.

```
function playNextNote() {  
  playNote(melody[step]);  
  step = (step + 1) % melody.length;  
  setTimeout(playNextNote, 125);  
}
```

C5. Collecting a row of coins produces a horrible crackle rather than a pleasant run of notes. Two [3 marks]
separate things are wrong.

```
for (const coin of touchedCoins) {  
  playCoinSound(audio.currentTime); // master gain is 1.0  
}
```

Section D — Build it

Start from your game, or from `code/01-one-sound-and-the-click.html`.

- ☐ **Checkpoint 1.** Create the context and a master gain at 0.3, and add the `resume()` unlock on first input. Print `audio.state` on screen — keep it there all lesson.
- ☐ **Checkpoint 2.** Write one `beep(freq, when, length)` with an envelope. Then make a version with no envelope and put them on two keys. Listen to both several times.
- ☐ **Checkpoint 3.** Build a **coin**: two short sine notes, the second a few semitones above the first, using the semitone formula.
- ☐ **Checkpoint 4.** Build a **jump**: a square wave sweeping upwards, using `frequency.linearRampToValueAtTime`. Then a **laser** by sweeping downwards instead. Same code, one number different.
- ☐ **Checkpoint 5.** Build a noise buffer, and from it an **explosion** (lowpass filter sweeping down) and a **footstep** (very short envelope). Put the filter frequency on a slider and find the value you like.
- ☐ **Checkpoint 6.** Add random pitch variation to every sound — multiply the frequency by `0.94 + Math.random() * 0.12`. Play the same sound twenty times in a row before and after.
- ☐ **Checkpoint 7.** Wire the sounds into your game: one per event. Add a count of currently-playing sounds on screen, and a cap.
- ☐ **Checkpoint 8.** Add a four-note looping bass line with the look-ahead scheduler. Then make the game deliberately busy (hold a key that spawns a hundred particles) and check the music stays in time.

IF THERE IS NO SOUND

In this order: is `audio.state "running"` ? Is the master gain

above 0? Is the oscillator connected to the master, and the master to

`audio.destination` ? Is the computer's own volume up? Four checks, in that order, finds it almost every time.</p>

Section E — Design challenge

NO RIGHT ANSWER

Marked on reasoning, and on naming the cost.

E1. Your sounds are built from parameters: shape, start frequency, end frequency, attack, decay. What would you gain by putting them all in one table, as you did with tiles in lesson 5? What would you lose?

E2. Eight coins are collected in one frame. Eight sounds is a mess; one is a lie. What *should* happen? Think about what the player is meant to learn from the sound.

E3. Design footsteps: never identical twice, never machine-regular, and silent the instant the player stops. What varies, by how much, and what decides *when* a step happens?

E4. The hard one. The music should become more intense as health drops — changing, not switching tracks. Name three musical things you could change at runtime, and say which would be audible without becoming annoying.

Stretch goals

1. All sounds in one data table.
2. Random pitch variation everywhere. One line, big improvement.
3. A clickable 16-step drum grid.
4. Two-layer music: a bass line always, a melody that fades in when in danger.
5. An `AnalyserNode` drawing the live waveform on your canvas.